Vibe Fuzzing— Building an AI-guided Fuzzing Engine for Subagents

Using Graphs to Fuzz the Entire Planet

Sam Lovejoy

October 12, 2026

Freely available from https://github.com/dinosaurlover38/tech-talks under a CC-BY-SA-NC-4 license.

Who am I?



- I’m a security researcher at IBM X-Force Offensive Research (XOR), we’re the exploit dev group behind X-Force’s Adversary Simulation group
- This project is one I built to do long-term vulnerability research, mostly against Windows



- If you find this research interesting, follow me on twitter/X for updates **@_dinolover38**

XOR
X-FORCE OFFENSIVE RESEARCH

Inspiration



There's been so many AIxFuzzing crossover episodes, but many of them fail to address the core, systemic limitations that this technology has:

1. Fragile inputs tend to break (e.g. xml)
2. No concept of program state or state machines
3. No concept of program semantics
4. Rarely useful outside of memory corruption
5. Expensive to build and maintain

In other words, fuzzing was designed to solve 2013 problems, not 2026 ones.



Inspiration



There's been so many AIxFuzzing crossover episodes, but many of them fail to address the core, systemic limitations that this technology has:

1. Fragile inputs tend to break (e.g. xml)
2. No concept of program state or state machines
3. No concept of program semantics
4. Rarely useful outside of memory corruption
5. Expensive to build and maintain

population

In other words, fuzzing was designed to solve 2013 problems, not 2026 ones.

Inspiration

There's been so many AIxFuzzing crossover episodes, but many of them fail to address the core, systemic limitations that this technology has:

1. Fragile inputs tend to break (e.g. xml)
2. No concept of program state or state machines
3. No concept of program semantics
4. Rarely useful outside of memory corruption
5. Expensive to build and maintain



In other words, fuzzing was designed to solve 2013 problems, not 2026 ones.

Inspiration

There's been so many AIxFuzzing crossover episodes, but many of them fail to address the core, systemic limitations that this technology has:

1. Fragile inputs tend to break (e.g. xml)
2. No concept of program state or state machines
3. No concept of program semantics
4. Rarely useful outside of memory corruption
5. Expensive to build and maintain



In other words, fuzzing was designed to solve 2013 problems, not 2026 ones.

Inspiration

There's been so many AIxFuzzing crossover episodes, but many of them fail to address the core, systemic limitations that this technology has:

1. Fragile inputs tend to break (e.g. xml)
2. No concept of program state or state machines
3. No concept of program semantics
4. Rarely useful outside of memory corruption
5. Expensive to build and maintain



In other words, fuzzing was designed to solve 2013 problems, not 2026 ones.

Inspiration

There's been so many AIxFuzzing crossover episodes, but many of them fail to address the core, systemic limitations that this technology has:

1. Fragile inputs tend to break (e.g. xml)
2. No concept of program state or state machines
3. No concept of program semantics
4. Rarely useful outside of memory corruption
5. Expensive to build and maintain



In other words, fuzzing was designed to solve 2013 problems, not 2026 ones.

Inspiration



There's been so many AIxFuzzing crossover episodes, but many of them fail to address the core, systemic limitations that this technology has:

1. Fragile inputs tend to break (e.g. xml)
2. No concept of program state or state machines
3. No concept of program semantics
4. Rarely useful outside of memory corruption
5. Expensive to build and maintain



In other words, fuzzing was designed to solve 2013 problems, not 2026 ones.

Inspiration

There's been so many AIxFuzzing crossover episodes, but many of them fail to address the core, systemic limitations that this technology has:

1. Fragile inputs tend to break (e.g. xml)
2. No concept of program state or state machines
3. No concept of program semantics
4. Rarely useful outside of memory corruption
5. Expensive to build and maintain



In other words, fuzzing was designed to solve 2013 problems, not 2026 ones.

Inspiration

There's been so many AIxFuzzing crossover episodes, but many of them fail to address the core, systemic limitations that this technology has:

1. Fragile inputs tend to break (e.g. xml)
2. No concept of program state or state machines
3. No concept of program semantics
4. Rarely useful outside of memory corruption
5. Expensive to build and maintain



In other words, fuzzing was designed to solve 2013 problems, not 2026 ones.

Inspiration

There's been so many AIxFuzzing crossover episodes, but many of them fail to address the core, systemic limitations that this technology has:

1. Fragile inputs tend to break (e.g. xml)
2. No concept of program state or state machines
3. No concept of program semantics
4. Rarely useful outside of memory corruption
5. Expensive to build and maintain



In other words, fuzzing was designed to solve 2013 problems, not 2026 ones.

Inspiration

There's been so many AIxFuzzing crossover episodes, but many of them fail to address the core, systemic limitations that this technology has:

1. Fragile inputs tend to break (e.g. xml)
2. No concept of program state or state machines
3. No concept of program semantics
4. Rarely useful outside of memory corruption
5. Expensive to build and maintain



In other words, fuzzing was designed to solve 2013 problems, not 2026 ones.

Making a Fuzzer for a Subagent



Vibe Fuzzing is the practice of delegating a fuzzing engine's control plane to an LLM to guide input generation in the data plane. This is most practically done as a subagent,

- The main assistant determines when fuzzing is a rational step forward.
- A subagent uses source code, coverage data, decompiler text, patch notes, engine state and so forth to control input generation.

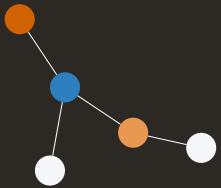
The implementation is called *vf3*— the very fast, very fun, vibe fuzzer.
It's almost like an SDK for building AI-powered fuzzers, and can fuzz anything from PNG images to the Windows NT Kernel!



The Data Plane

Syntactic Fuzzing– *Fragile Inputs tend to Break*

- vf3 builts its inputs from graphs, each graph is a Markov Chain, with each node implementing some function to build an input, these functions are:
  - Bytes objects
  - Python functions
  - Nested grammars
  - Engine SDK functions



$$T = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} N_1 \\ N_2 \\ N_3 \\ N_4 \\ N_5 \end{bmatrix}$$

Syntactic Fuzzing– *Fragile Inputs tend to Break*

- vf3 builts its inputs from graphs, each graph is a Markov Chain, with each node implementing some function to build an input, these functions are:
  - Bytes objects
  - Python functions
  - Nested grammars
  - Engine SDK functions



$$T = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} N_1 \\ N_2 \\ N_3 \\ N_4 \\ N_5 \end{bmatrix}$$

Syntactic Fuzzing– *Fragile Inputs tend to Break*

- vf3 builts its inputs from graphs, each graph is a Markov Chain, with each node implementing some function to build an input, these functions are:
  - Bytes objects
  - Python functions
  - Nested grammars
  - Engine SDK functions



$$T = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} N_1 \\ N_2 \\ N_3 \\ N_4 \\ N_5 \end{bmatrix}$$

Syntactic Fuzzing– *Fragile Inputs tend to Break*

- vf3 builts its inputs from graphs, each graph is a Markov Chain, with each node implementing some function to build an input, these functions are:
  - Bytes objects
  - Python functions
  - Nested grammars
  - Engine SDK functions



$$T = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} N_1 \\ N_2 \\ N_3 \\ N_4 \\ N_5 \end{bmatrix}$$

Syntactic Fuzzing– *Fragile Inputs tend to Break*



- The graph contains enough information to precisely fuzz individual fields without overriding or influencing others.
- Behind the scenes vf3 compiles the graphs into optimized tables.

Tool	Inputs/s	Mean Size	Throughput	vs Best Inputs/s	vs Best Throughput
vf3-1024	3.55M	192	651MB/s	0.0%	0.0%
vf3-1	3.32M	192	607MB/s	-6.8%	-6.8%
libprotobuf-mutator	2.3M	98	214 MB/s	-35.2%	-66.9%
Jackalope	1.70M	123	199 MB/s	-52.1%	-69.4%

Table: vf3's grammar engine versus other syntactic fuzzers generating HTTP/1.1 packets

Syntactic Fuzzing– *Fragile Inputs tend to Break*



- The graph contains enough information to precisely fuzz individual fields without overriding or influencing others.
- Behind the scenes vf3 compiles the graphs into optimized tables.

Tool	Inputs/s	Mean Size	Throughput	vs Best Inputs/s	vs Best Throughput
vf3-1024	3.55M	192	651MB/s	0.0%	0.0%
vf3-1	3.32M	192	607MB/s	-6.8%	-6.8%
libprotobuf-mutator	2.3M	98	214 MB/s	-35.2%	-66.9%
Jackalope	1.70M	123	199 MB/s	-52.1%	-69.4%

Table: vf3's grammar engine versus other syntactic fuzzers generating HTTP/1.1 packets

State Machines– *No Concept of Program State*



Syntactic fuzzing is not enough, even valid messages might be ignored or interpreted differently depending on the state of the program.

- Despite a majority of real-world software being stateful, there is no actively maintained state-aware fuzzing engine.

Machine Type	List of RFCs	OSS-Fuzz
Combinational logic	36.4%	45.4%
Finite State Machine	59.5%	26.2%
Stack Machine	3.3%	12.8%
Turing Machine	0.8%	15.6%

Table: Approx. state machine types of standards and software from Wikipedia and OSS-Fuzz.

State Machines– *No Concept of Program State*



Syntactic fuzzing is not enough, even valid messages might be ignored or interpreted differently depending on the state of the program.

- Despite a majority of real-world software being stateful, there is no actively maintained state-aware fuzzing engine.

Machine Type	List of RFCs	OSS-Fuzz
Combinational logic	36.4%	45.4%
Finite State Machine	59.5%	26.2%
higher order state machines { Stack Machine	3.3%	12.8%
higher order state machines { Turing Machine	0.8%	15.6%

Table: Approx. state machine types of standards and software from Wikipedia and OSS-Fuzz.

Turing Model Example



Imagine the POSIX system call *open*

```
int fd = open(char * path, int flags, int mode)
```



Turing Model Example



Imagine the POSIX system call *open*

```
int fd = open(char * path, int flags, int mode)
```

file descriptors are context
to track between system calls



Turing Model Example



Imagine the POSIX system call *open*

```
int fd = open(char * path, int flags, int mode)
```

file descriptors are context
to track between system calls

```
void * map = mmap(void * addr, size_t len, int prot, int flags, int fd, off_t offset)
```



Turing Model Example



Imagine the POSIX system call *open*

```
int fd = open(char * path, int flags, int mode)
```

file descriptors are context
to track between system calls

```
void * map = mmap(void * addr, size_t len, int prot, int flags, int fd, off_t offset)
```

Turing Model Example



Imagine the POSIX system call *open*

```
int fd = open(char * path, int flags, int mode)
```

file descriptors are context
to track between system calls

tracking only file descriptors
would be a *Stack Machine*

```
void * map = mmap(void * addr, size_t len, int prot, int flags, int fd, off_t offset)
```



Turing Model Example



Imagine the POSIX system call *open*

```
int fd = open(char * path, int flags, int mode)
```

file descriptors are context to track between system calls

tracking only file descriptors would be a *Stack Machine*

```
void * map = mmap(void * addr, size_t len, int prot, int flags, int fd, off_t offset)
```

```
int ok = mprotect(void * addr, size_t len, int prot)
```

Turing Model Example



Imagine the POSIX system call *open*

```
int fd = open(char * path, int flags, int mode)
```

file descriptors are context
to track between system calls

tracking only file descriptors
would be a *Stack Machine*

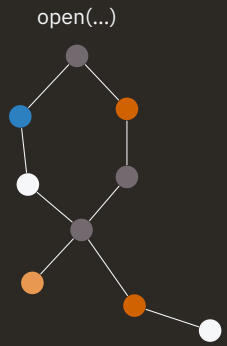
```
void * map = mmap(void * addr, size_t len, int prot, int flags, int fd, off_t offset)
```

```
int ok = mprotect(void * addr, size_t len, int prot)
```

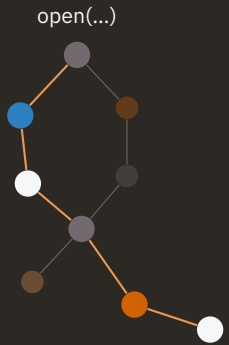
tracking two or more contexts
would be a *Turing Machine*



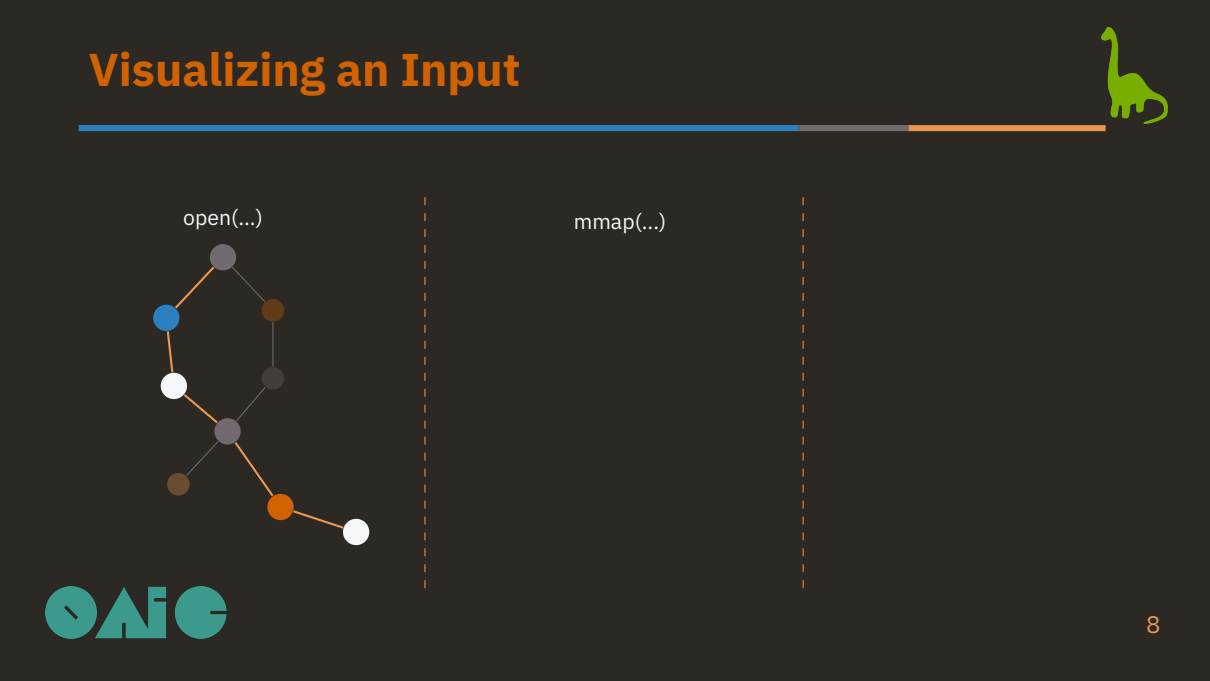
Visualizing an Input



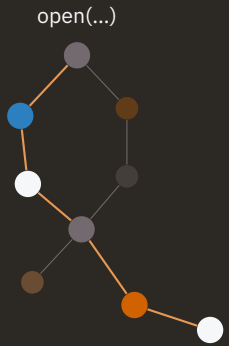
Visualizing an Input

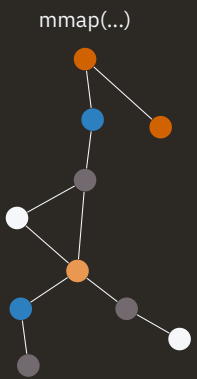


Visualizing an Input

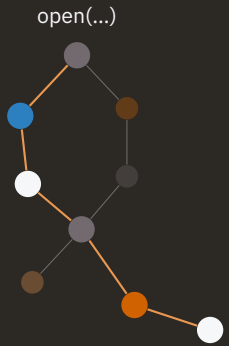


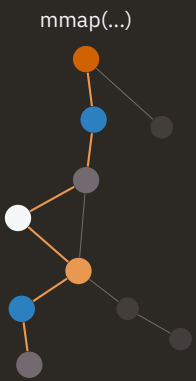
Visualizing an Input



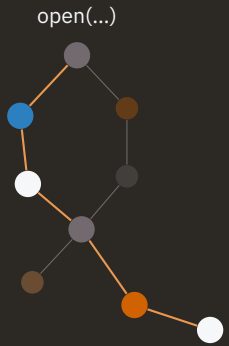


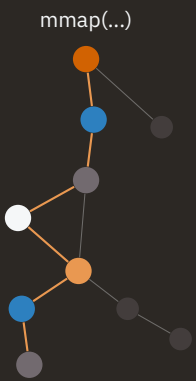
Visualizing an Input



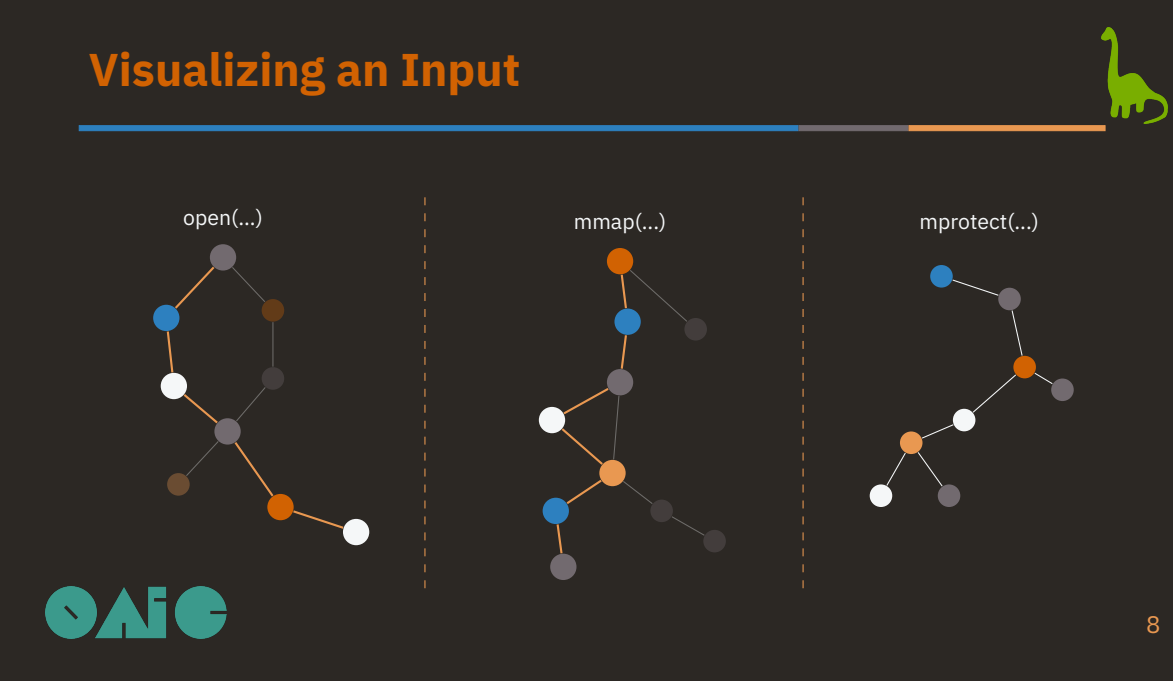


Visualizing an Input

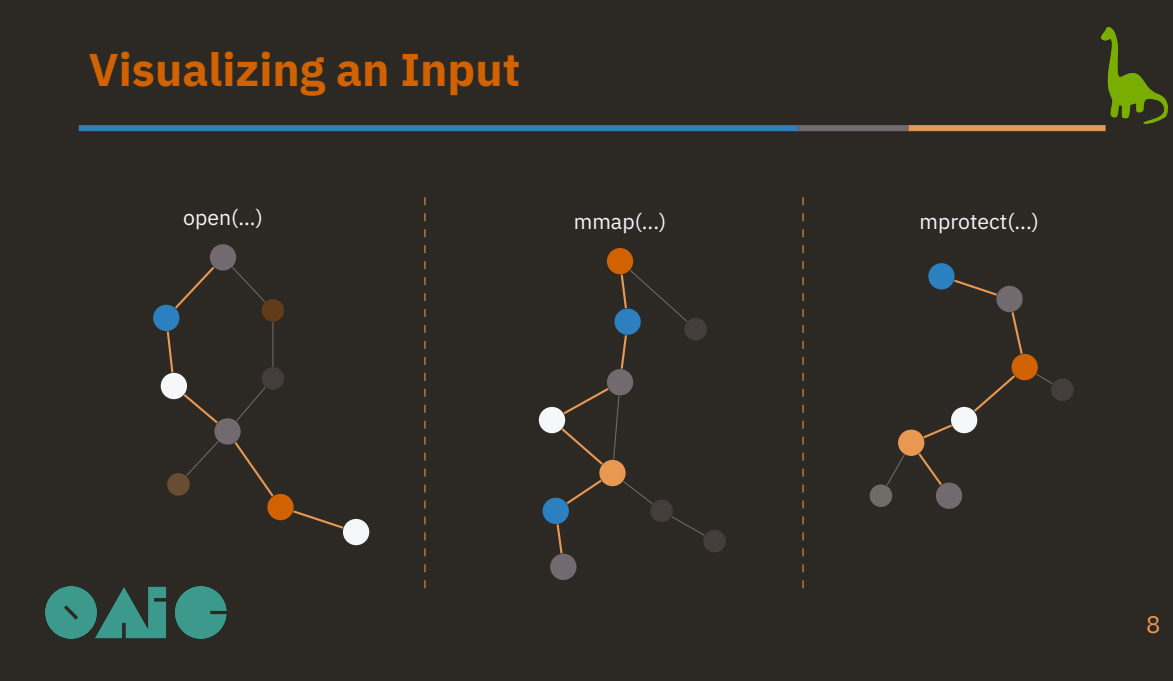




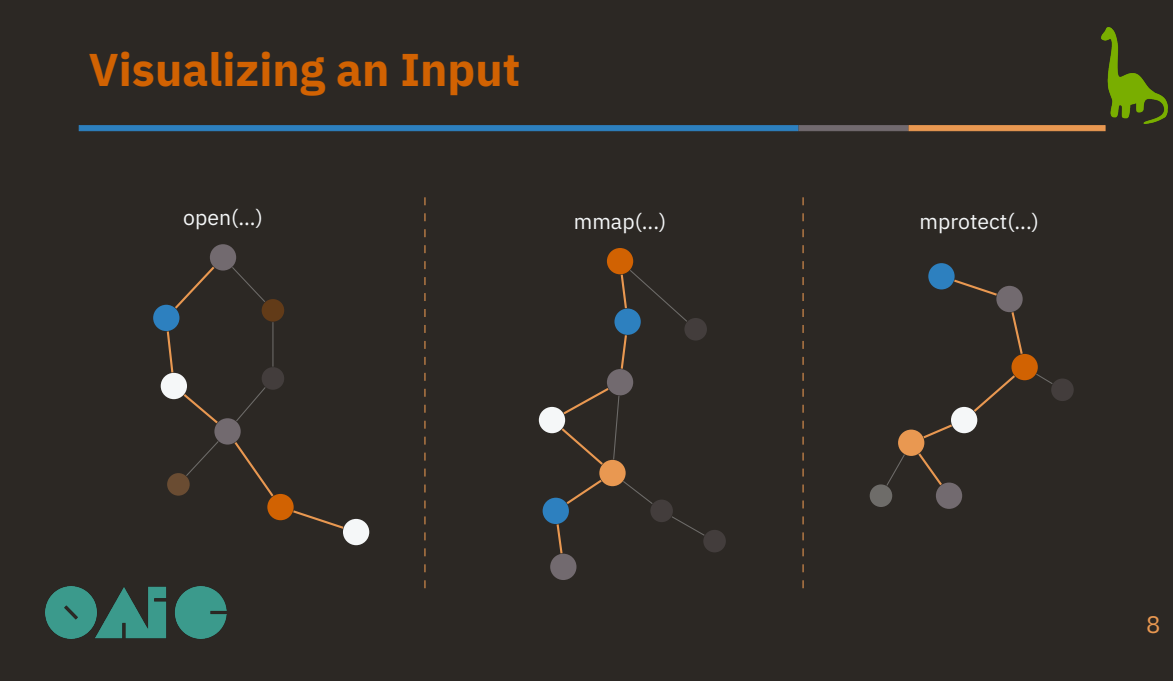
Visualizing an Input



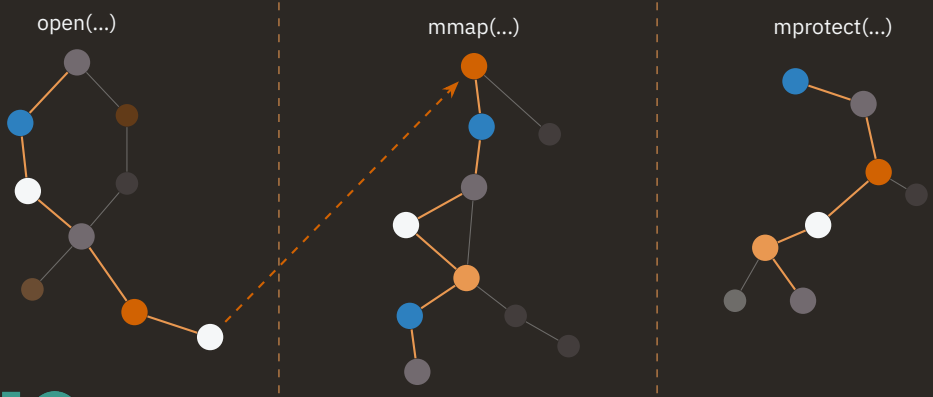
Visualizing an Input



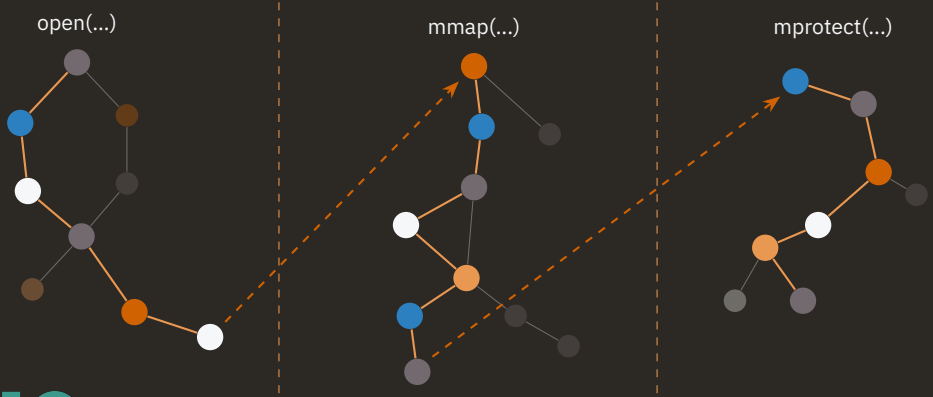
Visualizing an Input



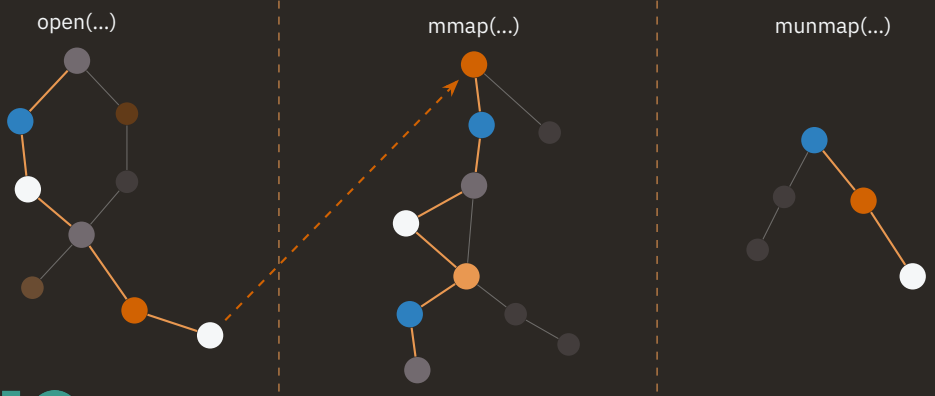
Visualizing an Input



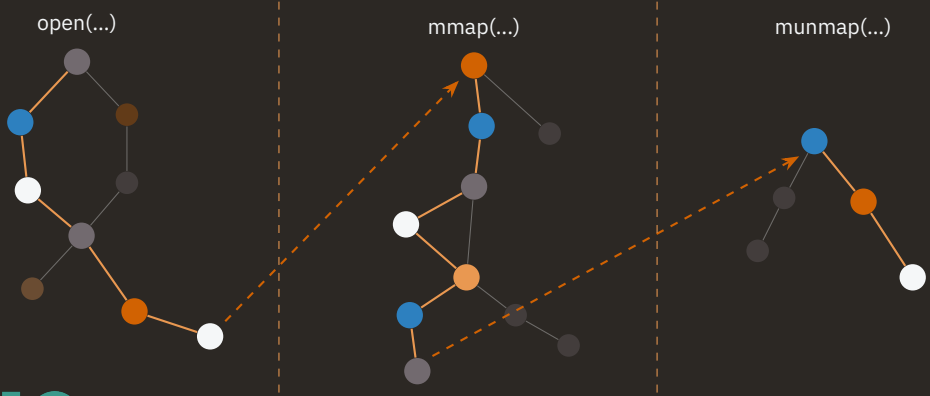
Visualizing an Input



Visualizing an Input



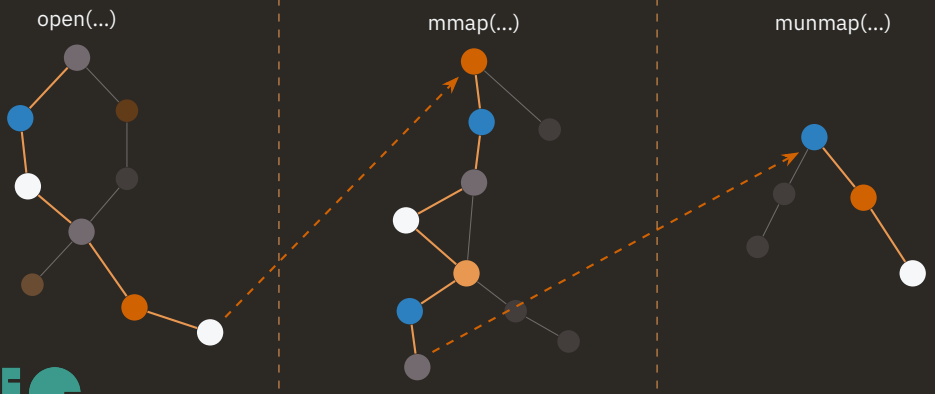
Visualizing an Input



Visualizing an Input



- The engine's job is to find paths throughout states & grammars!



The Control Plane

Fine Tuning Inputs- The Control Plane

- vf3's data plane forms one of the fastest syntactic engines, it's also the only engine with dedicated infrastructure for generalized advanced state machines.
- Under the hood, all of vf3's data plane is represented by graphs; thus, the engine's controls manifest as *graph edits*.
- Input machinery, the program harness, and controls coalesce into a *campaign*.



Fine Tuning Inputs- The Control Plane

- vf3's data plane forms one of the fastest syntactic engines, it's also the only engine with dedicated infrastructure for generalized advanced state machines.
- Under the hood, all of vf3's data plane is represented by graphs; thus, the engine's controls manifest as *graph edits*.
- Input machinery, the program harness, and controls coalesce into a *campaign*.



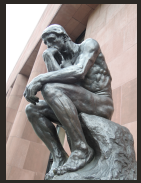
1. Fuzzer stuck

Fine Tuning Inputs- The Control Plane

- vf3's data plane forms one of the fastest syntactic engines, it's also the only engine with dedicated infrastructure for generalized advanced state machines.
- Under the hood, all of vf3's data plane is represented by graphs; thus, the engine's controls manifest as *graph edits*.
- Input machinery, the program harness, and controls coalesce into a *campaign*.



1. Fuzzer stuck



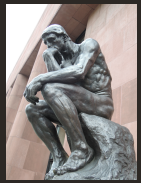
2. Ask large language friend

Fine Tuning Inputs- The Control Plane

- vf3's data plane forms one of the fastest syntactic engines, it's also the only engine with dedicated infrastructure for generalized advanced state machines.
- Under the hood, all of vf3's data plane is represented by graphs; thus, the engine's controls manifest as *graph edits*.
- Input machinery, the program harness, and controls coalesce into a *campaign*.



1. Fuzzer stuck



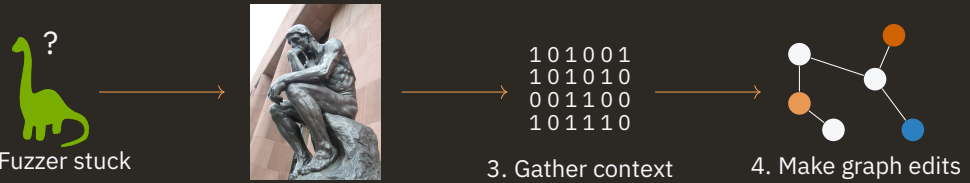
2. Ask large language friend

101001
101010
001100
101110

3. Gather context

Fine Tuning Inputs- The Control Plane

- vf3's data plane forms one of the fastest syntactic engines, it's also the only engine with dedicated infrastructure for generalized advanced state machines.
- Under the hood, all of vf3's data plane is represented by graphs; thus, the engine's controls manifest as *graph edits*.
- Input machinery, the program harness, and controls coalesce into a *campaign*.



When do I click the AI button?



AI is awesome, but it's really really slow. It can also be really expensive. We can mathematically determine ideal conditions to solicit help from AI.

$$D_{KL}(P||Q) = \sum_{x \in X} P(x) ln(\frac{P(x)}{Q(x)})$$

- This equation is called Kullback-Leibler (KL) divergence, it measures how far one probability distribution *Q* is from a target distribution *P*.
- In our case, if the fuzzer how no idea what it's doing, each input we process will have uniform or near-uniform response. P(x) will look like 1/N for N inputs.

Prompting and Gathering Context

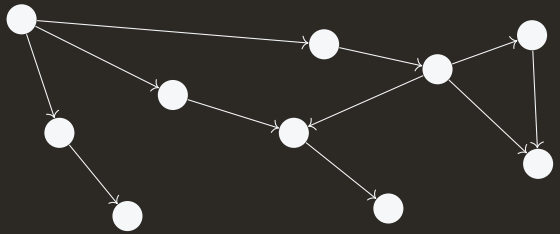


- vf3 routes its API calls through litellm, providing the model context via *tool calls*.
  - State & grammar info
  - Crash reports
  - Coverage data
  - Input history
- It's also possible to wire up MCP services, like GhidraMCP for additional context.
- In normal usage, vf3 asks the LLM how the campaign can be improved, but at any point one can interrupt fuzzing and invoke operations directly:
  - Route to line of code
  - Mutate fields
  - Mine new grammar
  - Fuzz variable
  - Fuzz patch
  - Modify routing algorithms



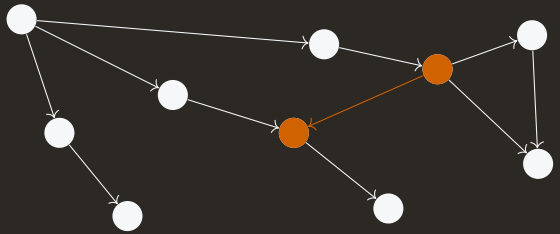
Targeting Specific Code & Patches

- To target specific code (from source/decompiler), the LLM *subgraphs* from an existing grammar at some state.
- Each field in our grammar has dedicated contextual mutations, for example: lengths, offsets, strings, cookies, encodings, etc.
  - Mutations can even carry payloads like SQL or prompt injection!



Targeting Specific Code & Patches

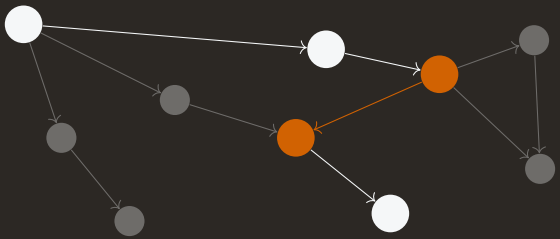
- To target specific code (from source/decompiler), the LLM *subgraphs* from an existing grammar at some state.
- Each field in our grammar has dedicated contextual mutations, for example: lengths, offsets, strings, cookies, encodings, etc.
  - Mutations can even carry payloads like SQL or prompt injection!



Targeting Specific Code & Patches



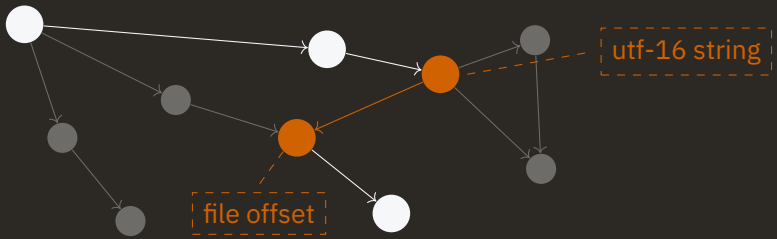
- To target specific code (from source/decompiler), the LLM *subgraphs* from an existing grammar at some state.
- Each field in our grammar has dedicated contextual mutations, for example: lengths, offsets, strings, cookies, encodings, etc.
  - Mutations can even carry payloads like SQL or prompt injection!



Targeting Specific Code & Patches

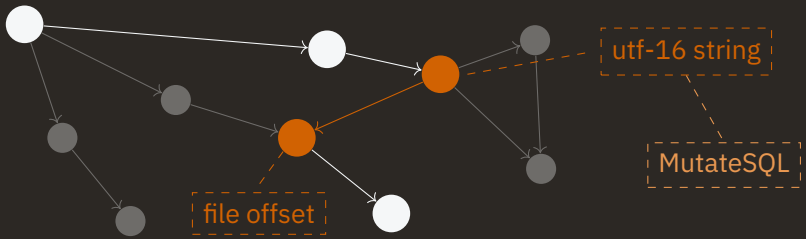


- To target specific code (from source/decompiler), the LLM *subgraphs* from an existing grammar at some state.
- Each field in our grammar has dedicated contextual mutations, for example: lengths, offsets, strings, cookies, encodings, etc.
  - Mutations can even carry payloads like SQL or prompt injection!



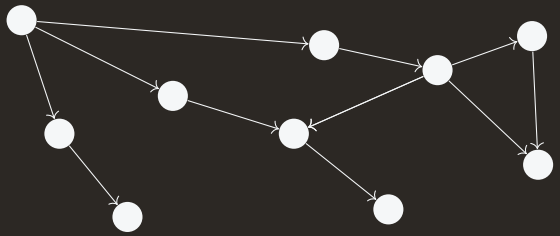
Targeting Specific Code & Patches

- To target specific code (from source/decompiler), the LLM *subgraphs* from an existing grammar at some state.
- Each field in our grammar has dedicated contextual mutations, for example: lengths, offsets, strings, cookies, encodings, etc.
  - Mutations can even carry payloads like SQL or prompt injection!



Mining New Grammars & Inputs

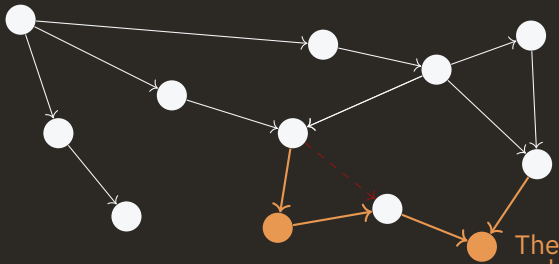
- Oftentimes, especially in proprietary code, vendors will extend specifications.
- Many standards have explicit carveouts for this like DHCP, Wi-Fi, and RADIUS.
- AI can adapt our syntaxes dynamically to probe these extensions.



Mining New Grammars & Inputs



- Oftentimes, especially in proprietary code, vendors will extend specifications.
- Many standards have explicit carveouts for this like DHCP, Wi-Fi, and RADIUS.
- AI can adapt our syntaxes dynamically to probe these extensions.



The LLM can modify add/remove nodes and edges from grammars!



Bigger Better Bugs

Moving Past Memory Corruption



- Vulnerabilities are much more complex than memory corruption, there needs to be more general systems to detect such issues.
- vf3 has experimental SDK support for logic bugs.

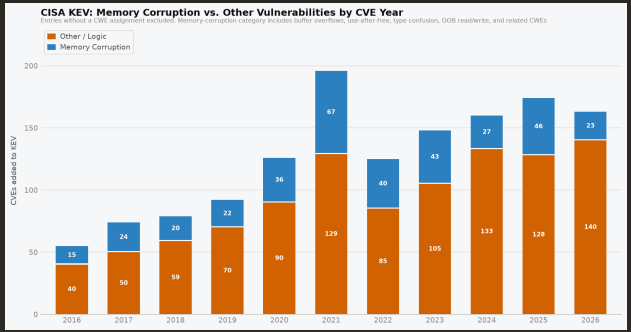


Figure: CISA KEV catalog vulnerabilities per year as memory corruption or logic issues.

Moving Past Memory Corruption



- Vulnerabilities are much more complex than memory corruption, there needs to be more general systems to detect such issues.
- vf3 has experimental SDK support for logic bugs.

```python
import numpy as np

def my_harness(data: np.ndarray, size: int
               features: np.ndarray) -> bool:
    """
    Process 'size' bytes in 'data'.
    Populate the feature space 'features'.
    Returning False from this function
    will engage vf3's reproducer.
    """
    # e.g. socket.send(data[:size])
    return True
```



Cost Complexity and Tokens



- vf3 has backwards compatibility with both AFL++ and libfuzzer harnesses.
- Excluding the reproducer, open-source models control the engine well.

Model	Size	Correctness (N = 100)
Granite 4-1:30b	17G	49%
Deepseek-r1:14b	9G	4%
qwen-3.8	17G	90%
Sonnet 4.6	cloud	99%

Table: Performance of various models in patch gapping N-days in Apache httpd.



Concluding Thoughts

Recap



In short, classical fuzzing is not equipped to handle all of 2026's problems,

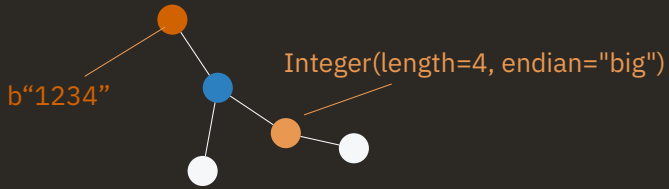
1. Fragile inputs tend to break
2. No concept of program state
3. No concept of program semantics
4. Limited applications outside of memory corruption
5. Expensive to build and maintain

Recap



In short, classical fuzzing is not equipped to handle all of 2026's problems,

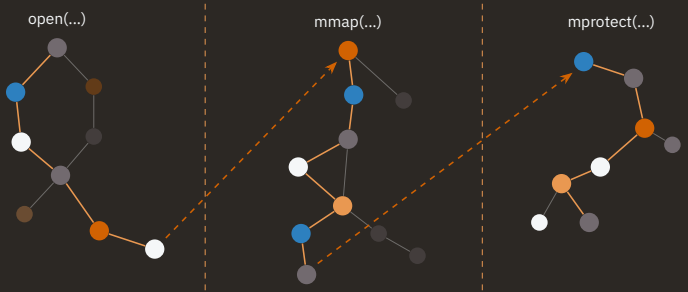
1. Fragile inputs tend to break
2. No concept of program state
3. No concept of program semantics
4. Limited applications outside of memory corruption
5. Expensive to build and maintain



Recap

In short, classical fuzzing is not equipped to handle all of 2026's problems,

1. ~~Fragile inputs tend to break~~
2. No concept of program state
3. No concept of program semantics
4. Limited applications outside of memory corruption
5. Expensive to build and maintain



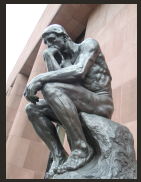
Recap

In short, classical fuzzing is not equipped to handle all of 2026's problems,

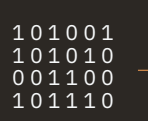
1. ~~Fragile inputs tend to break~~
2. ~~No concept of program state~~
3. No concept of program semantics
4. Limited applications outside of memory corruption
5. Expensive to build and maintain



1. Fuzzer stuck



2. Ask large language friend



3. Gather context



4. Make graph edits

Recap



In short, classical fuzzing is not equipped to handle all of 2026's problems,

1. ~~Fragile inputs tend to break~~
2. ~~No concept of program state~~
3. ~~No concept of program semantics~~
4. Limited applications outside of memory corruption
5. Expensive to build and maintain

```
def my_harness(data: np.ndarray, size: int
               features: np.ndarray) -> bool:
    """
    Process 'size' bytes in 'data'...
    """
    # do stuff
    return True
```



Recap



In short, classical fuzzing is not equipped to handle all of 2026's problems,

1. ~~Fragile inputs tend to break~~
2. ~~No concept of program state~~
3. ~~No concept of program semantics~~
4. ~~Limited applications outside of memory corruption~~
5. Expensive to build and maintain

Model	Size	Correctness (N = 100)
Granite 4-1:30b	17G	49%
Deepseek-r1:14b	9G	4%
qwen-3.8	17G	90%
Sonnet 4.6	cloud	99%

Recap



In short, classical fuzzing is not equipped to handle all of 2026's problems,

1. ~~Fragile inputs tend to break~~
2. ~~No concept of program state~~
3. ~~No concept of program semantics~~
4. ~~Limited applications outside of memory corruption~~
5. ~~Expensive to build and maintain~~

Vibe fuzzing offers a robust, cost-effective means for long-term testing and VR.

Thank yous!



Dustin Heywood
Valentina Palmiotti

OAIC Organization Crew

vf3 release w/ benchmarks coming November 2026 (@_dinolover38 for updates)

